Shortest-Path Planning in a Large-Scale Road Network: Dijkstra vs A*

Author: Shourya Sharma
Admission number: 25JE1162

1. Summary

This report compares Dijkstra's algorithm and A* on subgraphs of the San Francisco Bay Area road network from the 9th DIMACS Implementation Challenge. Both algorithms were implemented from scratch in C++ with an adjacency-list graph and a binary min-heap priority queue, and no library shortest-path routines were used. Experiments ran on five graph sizes (1,000 to 50,000 vertices) with five source-destination pairs per size.

The main findings are:

- Dijkstra and A* returned identical shortest-path costs at every graph size.
- A* explored 29% to 55% fewer vertices than Dijkstra (about 41% fewer on average across the five sizes).
- Execution times were very small (0.05 ms to 3.08 ms per query), and A* was only slightly faster in wall-clock time, probably because each vertex it touches needs an extra haversine computation.

---

2. Dataset and Graph Construction (Parts A and B)

Item	Details
Dataset	USA road network, San Francisco Bay Area (BAY), distance graph
Source	9th DIMACS Implementation Challenge, Shortest Paths (2005–2006): http://www.diag.uniroma1.it/challenge9/download.shtml
Files used	USA-road-d.BAY.gr (arcs and distance weights) and USA-road-d.BAY.co (node coordinates)
Full size	321,270 vertices, 800,172 directed arcs

Vertex	A road intersection or road endpoint
Edge	A directed road segment between two vertices
Edge weight	Road distance from the distance graph (approximately 0.1 m units, see Section 6, Q9)

The full graph is cut into smaller graphs by a breadth-first search from a fixed seed vertex (vertex 0). The first N vertices reached are kept, only arcs with both endpoints inside the set are retained, and the vertices are renumbered 0 to N-1. The subgraphs are therefore nested, with the 1,000-vertex graph contained in the 5,000-vertex graph, and so on.

Graph statistics (degrees are out-degrees of directed arcs):

Graph	Vertices	Directed arcs	Average out-degree	Maximum out-degree
Full	321,270	800,172	2.491	7
1,000	1,000	2,240	2.240	4
5,000	5,000	11,866	2.373	6
10,000	10,000	23,752	2.375	6
25,000	25,000	59,464	2.379	6
50,000	50,000	119,512	2.390	6

---

3. Implementation and Experimental Setup (Parts C, D and E)

- **Dijkstra:** binary min-heap (priority queue with greater<>), lazy deletion of stale heap entries, early stop when the destination is settled, and explicit path reconstruction through a parent array.
- **A*:** the same structure, ordering the heap by f(v) = g(v) + h(v), with h(v) = 7.02 x haversine(v, destination).
- **Unreachable pairs:** a pair with no path is reported as not found and skipped. The program prints how many pairs were skipped.
- **Metrics:** vertices explored is the number of vertices settled (popped and not stale). Edges examined is the number of outgoing edges looked at from settled vertices. Time is wall-clock time per query, including allocation of the distance and visited arrays.

- **Queries:** five reachable source–destination pairs per size, chosen uniformly at random with a fixed seed (mt19937, seed 42 plus the graph size), so the experiments are reproducible. The complete vertex sequence of every path is stored in results/paths.csv.
- **Machine:** Intel Core i5-12450HX, 16 GB RAM, Windows 11, g++ 15.2.0 (MinGW-W64) with -O2 -std=c++17.

---

4. Results (Part E)

Each value is the average over five queries. Costs are in dataset weight units (about 0.1 m), and the kilometre column is cost divided by 10,000.

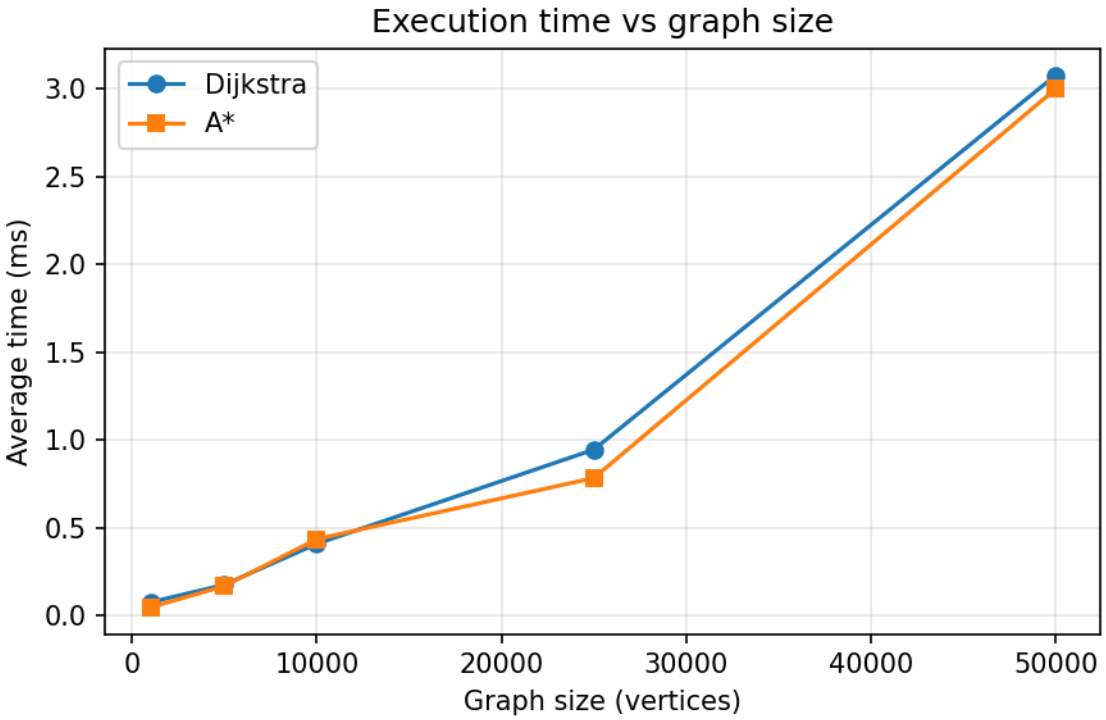
Graph size	Algorithm	Avg cost (units)	Avg cost (km)	Avg vertices explored	Avg edges examined	Avg time (ms)	Avg path length
1,000	Dijkstra	101,501.80	10.15	382.0	861.0	0.08	25.8
1,000	A*	101,501.80	10.15	173.8	397.8	0.05	25.8
5,000	Dijkstra	67,854.20	6.79	1,612.4	4,058.0	0.18	54.0
5,000	A*	67,854.20	6.79	964.6	2,451.0	0.17	54.0
10,000	Dijkstra	199,976.60	20.00	4,303.8	10,231.2	0.41	80.6
10,000	A*	199,976.60	20.00	3,074.0	7,352.4	0.43	80.6
25,000	Dijkstra	248,740.60	24.87	11,364.0	27,107.6	0.94	126.2
25,000	A*	248,740.60	24.87	6,159.2	14,772.6	0.78	126.2
50,000	Dijkstra	683,912.00	68.39	34,515.0	82,382.4	3.08	277.2
50,000	A*	683,912.00	68.39	22,782.8	54,652.8	3.00	277.2

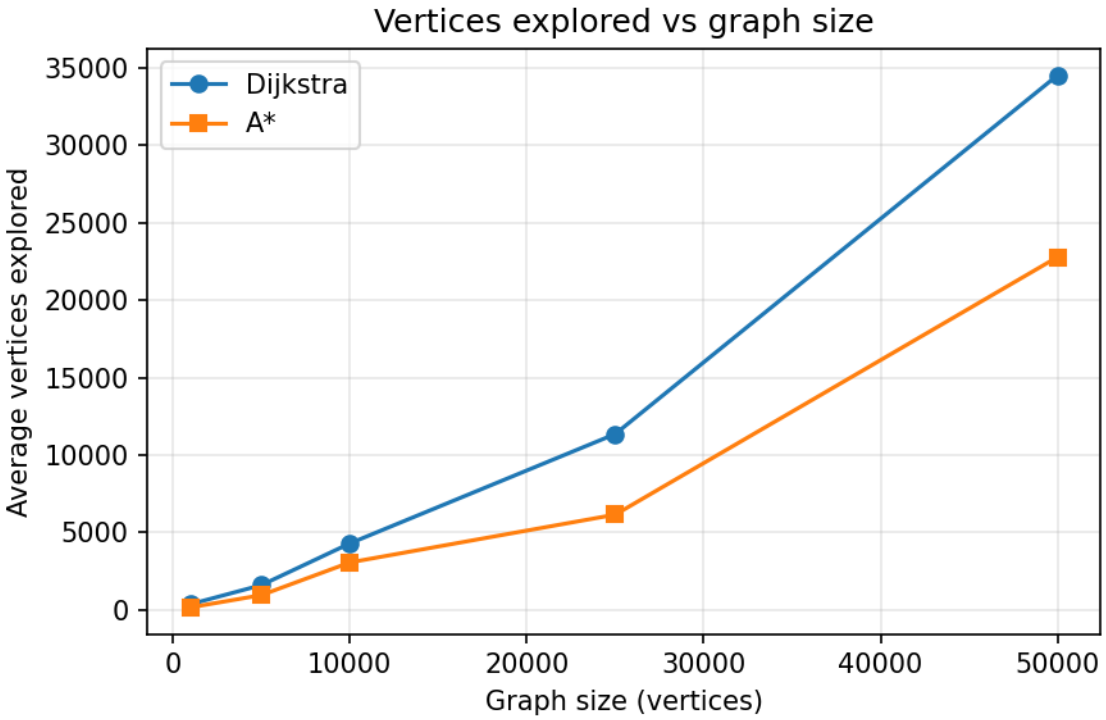
A* relative to Dijkstra:

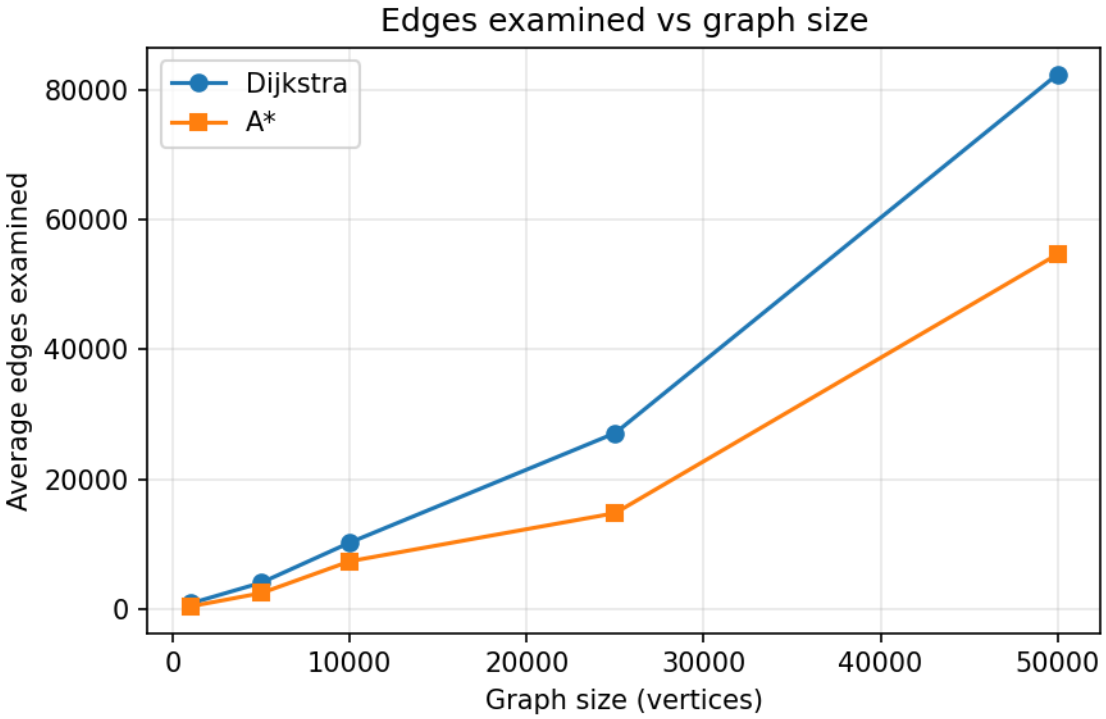
Graph size	Vertices explored (A*/Dijkstra)	Reduction in vertices explored	Edges examined (A*/Dijkstra)	Time (A*/Dijkstra)

1,000	0.455	54.5%	0.462	0.63
5,000	0.598	40.2%	0.604	0.94
10,000	0.714	28.6%	0.719	1.05
25,000	0.542	45.8%	0.545	0.83
50,000	0.660	34.0%	0.663	0.97

The plots (time.png, explored.png, edges.png and time_loglog.png) are in results/plots/.







5. Observations

1. **Identical optimal costs.** Dijkstra and A* give the same average cost and the same average path length at every graph size.

2. **A* explores fewer vertices at every size.** The reduction ranges from 28.6% (10,000 vertices) to 54.5% (1,000 vertices), with a mean of about 41%.
3. **Edges examined follow vertices explored.** The ratio of edges examined to vertices explored is between about 2.2 and 2.5 for both algorithms, which matches the average out-degree of the graph (2.2 to 2.4).
4. **The A* advantage does not show a clear trend with size.** The reduction is 54.5%, 40.2%, 28.6%, 45.8% and 34.0% across the five sizes. Each size uses a different set of five random pairs, so the differences between sizes may come from the particular pairs chosen.
5. **Time grows with graph size.** For Dijkstra, average time rises from 0.08 ms to 3.08 ms (about 38 times) while the graph grows 50 times. The average number of vertices explored rises about 90 times over the same range.
6. **Larger graphs mean farther-apart endpoints.** Average path cost grows from 10.2 km at 1,000 vertices to 68.4 km at 50,000 vertices, so random pairs on larger graphs are farther apart and the search covers a larger area.
7. **A* is only marginally faster in wall-clock time.** A* is 37% faster at 1,000 vertices, 6% faster at 5,000, 17% faster at 25,000 and 3% faster at 50,000, and about 5% slower at 10,000. All of these differences are fractions of a millisecond and may be within measurement noise.
8. **The graphs are very sparse.** Average out-degree is about 2.2 to 2.4 at every size, so the number of edges is roughly 2.4 times the number of vertices.
9. **Some pairs are unreachable.** BFS subgraphs are reachable from the seed but not necessarily strongly connected, so some random pairs have no path. These were detected and skipped.

---

6. Answers to Part F

Q1. Why is the dataset suitable for shortest-path analysis?

The BAY road network is real data, with 321,270 vertices and 800,172 arcs, which is large enough to cut out subgraphs from 1,000 to 50,000 vertices from one consistent source. Its edges carry positive distance weights, as Dijkstra requires. Every vertex has coordinates, which makes a geometric A* heuristic possible. It is sparse and has the structure of a real mobility network (intersections joined by road segments), so it is a realistic model for vehicle route planning. It

is also public, standard and well cited in the shortest-path literature.

Q2. How did you transform the raw dataset into a weighted graph?

1. Parsed the .gr file: lines of the form a u v w becomes directed arcs, and the p sp n m header gives the vertex and arc counts, which were checked against the number of arcs read.
2. Shifted the 1-indexed node IDs to 0-indexed.
3. Parsed the .co file and divided the integer coordinates by 10^6 to get longitude and latitude in degrees.
4. Built an adjacency list adj[u] = [(v, w), ...], discarding nothing that shortest-path computation needs.
5. Extracted subgraphs of 1,000, 5,000, 10,000, 25,000 and 50,000 vertices by BFS from vertex 0, kept the arcs inside each set, renumbered the vertices, and wrote them to CSV files.
6. Computed the minimum ratio of arc weight to haversine distance, which sets the A* heuristic scale.

Q3. Do Dijkstra and A* produce the same optimal path cost in your experiments?

Yes. The average cost and the average path length are identical at every graph size. This is expected: A* returns an optimal path whenever its heuristic is admissible and consistent (Q8). I also compared the cost of every individual pair, since averages can hide offsetting differences. Across all 25 pairs (5 per size) the costs were identical.

Q4. Which algorithm explores fewer vertices, and why?

A* explores fewer vertices at every size (28.6% to 54.5% fewer). Dijkstra orders vertices only by g(v), the distance from the source, so it expands in all directions in a region around the source until it reaches the destination. A* orders them by f(v) = g(v) + h(v), where h(v) estimates the remaining distance to the destination, so vertices that lead away from the destination are expanded later or never. The reduction is moderate rather than dramatic because the heuristic scale (7.02) is set by the weakest arc, while most arcs have a ratio of about 10 (see Q8), and because real roads are not straight lines.

Q5. How does execution time vary as the graph size increases?

Time grows with graph size for both algorithms. Dijkstra rises from 0.08 ms (1,000 vertices) to 3.08 ms (50,000 vertices), and A* from 0.05 ms to 3.00 ms. The growth is mainly driven by the number of vertices explored, which grows faster than the graph (about 90 times for a 50-times larger graph) because random pairs on a larger graph are farther apart. Time grew less than the explored count did, which may be due to fixed per-query costs and timer resolution at the smallest sizes; I did not isolate this. Times are sub-millisecond for

the small graphs and come from only five queries per size, so they are noisy.

Q6. Explain the effect of the graph's density on shortest-path computation.

Density determines how many edges are examined for each settled vertex. In this dataset the average out-degree is about 2.4 at every size, so the number of edges E is about 2.4 times the number of vertices V. The number of edges examined per explored vertex was between 2.2 and 2.5 in our runs, matching the average degree. In such a sparse graph, the running time is governed by V log V, and the work per vertex is small and almost constant.

In a denser graph, each settled vertex would require more edge examinations and more heap insertions, so the E log V term would dominate. In the extreme case of a dense graph (E close to V squared), a heap-based Dijkstra would be slower than a simple array-based version that runs in O(V squared). Because the average degree here barely changes between sizes, these experiments isolate the effect of size and do not vary density. The 1,000-vertex subgraph has a slightly lower degree (2.24), probably because BFS subgraphs lose arcs at their boundary.

Q7. What is the theoretical time complexity of your Dijkstra implementation?

With a binary heap and lazy deletion, each arc can cause at most one heap insertion, so the heap holds at most E + 1 entries. Each insertion and removal costs O(log E), and since E is at most V squared, log E is O(log V). The total is O((V + E) log V). Space is O(V + E). For this sparse graph, E = O(V), so the running time is O(V log V). Because the search stops when the destination is settled, the actual work depends on the number of vertices closer to the source than the destination, which is why the measured counts are well below V.

Q8. What heuristic did you use for A*? Is it admissible and/or consistent?

The heuristic is h(v) = 7.02 x haversine(v, destination), where haversine is the great-circle distance in metres between the coordinates of v and the destination. The factor 7.02 comes from the data: over all arcs of the full graph, the minimum of (arc weight) / (haversine distance of its endpoints) is 7.0284, and I rounded down to 7.02 to avoid floating-point problems at the boundary. The median ratio is 9.99, which is why the weights are approximately in units of 0.1 m.

Admissible. Every arc satisfies w(u, v) >= 7.0284 x haversine(u, v). Any path from v to the destination is a chain of arcs, so its cost is at least 7.0284 times the sum of the haversine lengths of its arcs. By the triangle inequality for great-circle distance, that sum is at least haversine(v, destination). The true remaining cost is therefore

at least 7.0284 x haversine(v, destination), which is at least h(v), so h never overestimates.

Consistent. For any arc (u, v), h(u) − h(v) = 7.02 x (haversine(u, dest) − haversine(v, dest)), which is at most 7.02 x haversine(u, v) by the triangle inequality, and that is at most w(u, v). So h(u) <= w(u, v) + h(v). Consistency means a vertex never needs to be re-expanded, so the implementation can mark vertices as settled the first time they are popped. Each subgraph's arcs are a subset of the full graph's arcs, so the same scale applies to all of them.

Trade-off. The scale is set by the weakest arc, so on a typical arc (ratio about 10) the heuristic recovers only about 70% of the distance information. A larger scale would prune more, but it would overestimate on the few low-ratio arcs, and optimality would no longer be guaranteed.

Q9. What difficulties did you face while converting the real-world dataset into a graph?

- **Indexing and units of the input files.** Node IDs are 1-indexed, and coordinates are integers in units of 10^-6 degrees, so both had to be converted before computing distances.
- **Edge weight units.** The DIMACS documentation describes distance weights as metres, but the weight-to-haversine ratio had a median of 9.99 (1st to 99th percentile of 9.98 to 10.01), so the weights are approximately in units of 0.1 m. I determined the unit empirically and did not find the cause of the discrepancy. All costs in this report are in those units.
- **Choosing a valid heuristic scale.** A raw haversine in metres would be far smaller than the real cost and would barely help A*, so the scale had to be measured from the data. The minimum ratio (7.0284) is an outlier compared with the typical value of 10.
- **Directed arcs.** The graph is directed, so degrees are out-degrees of arcs, and the number of arcs should not be read as a count of undirected road segments. Statistics had to be reported with that in mind.
- **Extracting subgraphs.** A BFS subgraph is a connected patch of road, but it is reachable from the seed rather than strongly connected, so some random pairs are unreachable. Arcs crossing the subgraph boundary are lost, so some shortest paths in the full graph are not available in a subgraph.
- **Nested subgraphs.** All sizes come from the same seed, so the smaller graphs are contained in the larger ones and the sizes are not independent samples.

Q10. Which algorithm would you recommend for large-scale IoV/UAV route planning?

I would recommend A* with an admissible geometric heuristic whenever positions are available, which is the case for vehicles and UAVs with GPS. In our experiments it returned exactly the same optimal costs as Dijkstra while exploring about 41% fewer vertices on average, and it needs no preprocessing. The saving matters most on large graphs and when many queries are answered, since each query touches fewer vertices.

The wall-clock advantage in our runs was small (3% to 37% faster at four sizes and about 5% slower at one), probably because each vertex touched needs a haversine calculation. A cheaper heuristic, such as Euclidean distance on projected coordinates, would reduce that cost. Dijkstra remains the better choice when coordinates are missing, when one source needs distances to many destinations at once, or when edge costs have no reliable geometric lower bound.

For dynamic costs, if congestion only increases the weights, the distance-based heuristic stays admissible. If conditions can make links cheaper than before, the heuristic scale must be recomputed from the new weights, or optimality can be lost. For networks much larger than the 50,000-vertex graphs tested here, speed-up techniques with preprocessing would be worth investigating, but I did not test them.

---

7. Limitations

- Each size uses only five random source-destination pairs, so the averages are sensitive to the particular pairs chosen, and the trends between sizes should be read cautiously.
- Times were measured from a single run per query with sub-millisecond values, so small timing differences are not reliable.
- All results come from one region (the Bay Area) and one seed for the subgraph extraction.
- The bonus task on dynamic edge costs was not attempted.

---

8. Reproducibility

The source code, run instructions, seeds, library versions and the dataset citation are in the project Readme. The commands run in order are preprocessing.py, then main.cpp (compiled with -O2 -std=c++17) with heuristic scale 7.02, then analysis.py. The per-query results are in results/results.csv and the complete paths are in results/paths.csv.

9. References

9th DIMACS Implementation Challenge, Shortest Paths (2005–2006).

Dataset